

AlwaysInstallElevated

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

AlwaysInstallElevated es una política de Windows que, si esta activada en las dos claves de registro (HKCU y HKLM), hace que cualquier paquete .msi se instale con privilegios de SYSTEM sin importar quien lo lance. Un atacante genera un .msi malicioso y lo instala para ejecutar código como SYSTEM.

La política existe para que usuarios sin privilegios puedan instalar software aprobado, pero abre un agujero enorme: el instalador corre como SYSTEM y el .msi puede hacer cualquier cosa. Es un vector limpio y fiable cuando esta activo. Caso del winprivesc-modelo.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). Se comprueba leyendo las dos claves de registro; WinPEAS lo marca explícitamente.

Cómo identificarla

- Ambas claves AlwaysInstallElevated valen 1 (HKCU y HKLM).
- WinPEAS resalta 'AlwaysInstallElevated set to 1'.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

reg query en ambas claves AlwaysInstallElevated (HKCU y HKLM); las dos deben ser 0x1.

Evidencia esperada: 0x1 en ambas claves. Tras instalar tu .msi, tu usuario aparece en el grupo Administrators.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.

5. Aplica una sola adaptación cuando haya evidencia de restricción.

6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Objetivo y contexto: Comprueba la clave de usuario. La escalada solo funciona si tanto HKCU como HKLM tienen AlwaysInstallElevated = 1; una sola no basta.

Resultado esperado: El valor (0x1 = activado). Repite la consulta en HKLM. Si ambas son 0x1, el vector aplica.

Capa 2: prueba documentada

```
msiexec /quiet /qn /i C:\Windows\Temp\pe.msi
```

Objetivo y contexto: Instala tu .msi en silencio. Con la política activa, msiexec lo ejecuta como SYSTEM, corriendo el payload que metiste dentro (crear admin o shell).

Resultado esperado: El payload se ejecuta como SYSTEM (p. ej. tu usuario nuevo aparece en 'net localgroup administrators'). Genera el .msi con la acción deseada de antemano.

Anatomía de los payloads

La primera prueba es `reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

0x1 en ambas claves. Tras instalar tu .msi, tu usuario aparece en el grupo Administrators.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Comprueba las dos claves de registro; deben valer 1 las dos.
- Genera un .msi malicioso (crea un admin o lanza tu shell).
- Transfiere y instálalo con msiexec en modo silencioso.
- Confirma privilegios de SYSTEM/admin.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Cualquiera de las dos claves (HKCU o HKLM) vale 0 o no existe.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si esta política esta activa, cualquier .msi se instala como SYSTEM: generas un instalador malicioso y listo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (always-install-elevated)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.